

0.110. ALPHABET_TITRE — les lettres des titres

`#include <ALPHABET_TITRE>` : `ALPHABET_TITRE[0]` est le A des titres — une lettre épaisse, avec une ombre grise —, pour écrire le nom d'un jeu sur son écran titre.

1. Ce qu'il faut comprendre

Ce que c'est : un **troisième alphabet**, pour les **titres** : des lettres **épaisses** (comme `ALPHABET_GRAS`) avec une **ombre** grise en bas à droite, qui leur donne du relief — le genre de lettres qu'on voit sur l'écran titre d'un jeu Game Boy. `ALPHABET_TITRE[0]` est le A, `ALPHABET_TITRE[25]` le Z. L'alphabet ordinaire ne change pas.

La ligne à écrire : `#include <ALPHABET_TITRE>`.

Comment trouver une lettre : le nombre entre crochets est sa **place dans l'alphabet**, en comptant à partir de 0. A = 0, B = 1, C = 2... T = 19. Pour écrire « TITRE » : T = 19, I = 8, R = 17, E = 4.

Chaque lettre est une tuile de 8 × 8, avec trois nuances : le fond (0), l'ombre (2) et le trait (3). On la pose avec `poser()`, comme n'importe quelle tuile, une case par lettre.

Ce qu'il coûte : **beaucoup**, environ **1 200 octets** dès la première lettre — les 26 dessins et le tableau qui les range partent ensemble. C'est fait pour un **titre**, pas pour tout le texte du jeu.

Ce sont nos lettres : elles sont calculées à partir de la police de l'atelier, dans le style des titres de la Game Boy, sans recopier celles d'aucun jeu.

Essaie : écris le nom de ton jeu. Une lettre par case : `poser(colonne, ligne, ALPHABET_TITRE[place])`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1384 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
main()         78 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
CopierTuiles   18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles         1120 o ← CopierTuiles — la police entière, et 26 tuiles dessinées
"TTITRE"       5 o ← texte() — un texte gravé
ALPHABET_TITRE_0 26 o ← poser() — une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
poser() × 5    60 o sur place, dans main() + ALPHABET_TITRE_0 (26 o)
```

```
texte() × 1    11 o sur place, dans main() + EcrireTexte (10 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <ALPHABET_TITRE> : les lettres des titres ----
// Le mot « TITRE » en lettres de titre – épaisses, avec une ombre –,
// et en dessous, le même mot en lettres ordinaires, pour comparer.

#include <ALPHABET_TITRE>    // ALPHABET_TITRE : 26 lettres de titre
#include <poser>             // poser() : une tuile sur une case du fond
#include <texte>             // texte() : le même mot, en lettres ordinaires
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

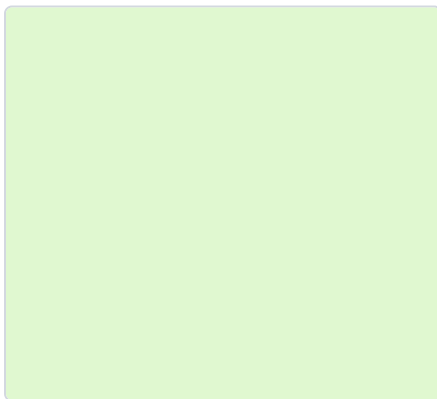
```
int main() {
    // ALPHABET_TITRE[place] : la place de la lettre dans l'alphabet, à partir de 0.
    //   A = 0, B = 1, C = 2, ... E = 4, ... I = 8, ... R = 17, ... T = 19
    poser(7, 6, ALPHABET_TITRE[19]);    // T
    poser(8, 6, ALPHABET_TITRE[8]);     // I
    poser(9, 6, ALPHABET_TITRE[19]);    // T
    poser(10, 6, ALPHABET_TITRE[17]);   // R
    poser(11, 6, ALPHABET_TITRE[4]);    // E

    texte(7, 9, "TITRE");                // le même mot, en lettres ordinaires

    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

TITRE en grandes lettres épaisses avec une ombre grise, et en dessous, TITRE en lettres ordinaires.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les deux T sont la même tuile de titre (63)
- ✓ les cinq lettres sont posées, toutes différentes du T sauf le second
- ✓ ce n'est pas le T ordinaire
- ✓ les lettres ont leur ombre grise (63 pixels gris)
- ✓ et en dessous, TITRE en lettres ordinaires

Ensuite : **0.111. Grand** — l'autre nom d'un Perso de 32 × 32 — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche
Game Boy en C++